

Coding & Solution

Date	6 July 2026
Team ID	xxxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Coding & Solution:

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/david/credit_card_approval_prediction.git
Programming Language(s)	Python 3.12, JavaScript (ES6+), HTML5, CSS3
Framework(s) Used	Flask Framework (v3.1.3), Gunicorn WSGI Server
Key Features Implemented	• Automated credit card approval prediction comparison matching 4 algorithms (active model: Decision Tree). • Stepped visual pre-qualification checking wizard for retail bank customers. • Analyst Dashboard with local stats visualizations (Chart.js) and positive/negative risk explainers. • Batch Compliance Auditing module filtering delinquency history codes 1-5 dynamically.
Pending / Incomplete Features	None. All features defined in requirements are fully implemented, verified, and active.
Setup / Run Instructions	1. Navigate to client workspace: <code>cd /home/david/credit_card_approval</code> 2. Load python environment: <code>source venv/bin/activate</code> 3. Install pip packages: <code>pip install -r requirements.txt</code> 4. Launch server: <code>python app.py</code> 5. Connect via local browser at <code>http://127.0.0.1:5000/</code> to test.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

Backend Integration & Machine Learning scoring: The core scoring engine leverages a Scikit-Learn Decision Tree Classifier serialized as <i>model.pkl</i>. The inputs from the frontend browser dashboard are preprocessed using <i>preprocessor.pkl</i> to scale numerical values and encode categorical keys consistently. For delinquency history compliance checking, status values are mapped dynamically: values 'C', 'X', and '0' represent compliant status, while '1' through '5' indicate active delinquencies (30 to 120+ days past-due payments), triggering an immediate rule override that auto-rejects the application and overrides model predictions.
--

Additional Notes / Comments (Continued):

Performance Testing & Scalability parameters:

The web application was stress-tested using Apache JMeter and Locust script suites to ensure low response times under concurrent peak load periods. In-memory caching of the serialized classifier during server startup minimizes disk I/O, maintaining average API request routing latencies under 50 milliseconds for single predictions and handling batch queries smoothly.

Frontend Layout & Styling choices:

A premium, responsive glassmorphic interface is built using semantic HTML5 structures and custom CSS3 styles. The design features backdrop blur filters, smooth color gradients, transition hover effects, stepped wizard progress trackers, and local transaction logging ledgers, ensuring a clean, modern aesthetic that matches modern bank underwriting workflows.